1. Indexes examined: **7, 11, 9**
   Value returned from search: **9**
2. Indexes examined: **6, 2, 4, 3**
   Value returned from search: **-1 (not found)**
3. {29, 17, 3, 94, 46, 8, -4, 12}
   First split: **{29, 17, 3, 94}{46, 8, -4, 12}**
   Second split: **{29, 17}{3, 94}{46, 8}{-4, 12}**
   Third split: **{29}{17}{3}{94}{46}{8}{-4}{12}**
   First merge: **{17, 29}{3, 94}{8, 46}{-4, 12}**
   Second merge: **{3, 17, 29, 94}{-4, 8, 12, 46}**
   Third merge: **{-4, 3, 8, 12, 17, 29, 46, 94}**
4.
   1.

```java
import java.util.Comparator;

Windsurf: Refactor | Explain
public class CaseInsensitiveComparator implements Comparator<String> {
    Windsurf: Refactor | Explain | Generate Javadoc | ×
    @Override
    public int compare(String s1, String s2) {
        return s1.compareToIgnoreCase(s2);
    }
}
```

   2.

```java
import java.util.Comparator;

Windsurf: Refactor | Explain
public class StringReverseComparator implements Comparator<String> {
    Windsurf: Refactor | Explain | Generate Javadoc | ×
    @Override
    public int compare(String s1, String s2) {
        // i and j start at last character of each string
        int i = s1.length() - 1;
        int j = s2.length() - 1;

        // compare characters from the right, stop when we reach first character
        while (i >= 0 && j >= 0) {
            if (s1.charAt(i) != s2.charAt(j)) { // if characters are different
                return s1.charAt(i) - s2.charAt(j);
            }
            // characters are equal, decrement i and j (move left one character)
            i--;
            j--;
        }

        return s1.length() - s2.length(); // if loop ends, shorter string comes first
    }
}
```

3.

```
import java.util.Comparator;

Windsurf: Refactor | Explain
public class LengthComparator implements Comparator<String> {
    Windsurf: Refactor | Explain | Generate Javadoc | ×
    @Override
    public int compare(String s1, String s2) {
        if (s1.length() != s2.length()) { // if strings aren't same length
            return s1.length() - s2.length(); // return difference
        }
        // strings are same length
        return s1.compareTo(s2); // sort alphabetically
    }
}
```

5. Initial: [5, 1, 7, 4, 9, 8, 0, 2, 3, 6]

    1.

    [0, 1, 7, 4, 9, 8, 5, 2, 3, 6]

    [0, 1, 2, 4, 9, 8, 5, 7, 3, 6]

    [0, 1, 2, 3, 9, 8, 5, 7, 4, 6]

    [0, 1, 2, 3, 4, 8, 5, 7, 9, 6]

    [0, 1, 2, 3, 4, 5, 8, 7, 9, 6]

    [0, 1, 2, 3, 4, 5, 6, 7, 9, 8]

    [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

    2.

    [1, 5, 7, 4, 9, 8, 0, 2, 3, 6]

    [1, 4, 5, 7, 9, 8, 0, 2, 3, 6]

    [1, 4, 5, 7, 8, 9, 0, 2, 3, 6]

    [0, 1, 4, 5, 7, 8, 9, 2, 3, 6]

    [0, 1, 2, 4, 5, 7, 8, 9, 3, 6]

    [0, 1, 2, 3, 4, 5, 7, 8, 9, 6]

    [0, 1, 2, 3, 4, 5, 6, 7, 8, 9]

    3.

[7, 1, 5, 4, 9, 8, 0, 2, 3, 6]

[7, 9, 5, 4, 1, 8, 0, 2, 3, 6]

[7, 9, 8, 4, 1, 5, 0, 2, 3, 6]

[7, 9, 8, 6, 1, 5, 0, 2, 3, 4]

[9, 7, 8, 6, 1, 5, 0, 2, 3, 4]

[9, 8, 7, 6, 1, 5, 0, 2, 3, 4]

[9, 8, 7, 6, 5, 1, 0, 2, 3, 4]

[9, 8, 7, 6, 5, 4, 0, 2, 3, 1]

[9, 8, 7, 6, 5, 4, 2, 0, 3, 1]

[9, 8, 7, 6, 5, 4, 2, 3, 0, 1]

[9, 8, 7, 6, 5, 4, 2, 3, 1, 0]

[9, 8, 7, 6, 5, 4, 3, 2, 1, 0]